

VisionGuard AI

Platform Overview & Setup Guide

This document provides an overview of VisionGuard AI — including features, tech stack, project structure, quick start guide, API overview, onboarding flow, and complete feature list.

Version	v2.0
Document Type	Platform Overview & Setup Guide
Platform	VisionGuard AI
Status	Final

```
<p align="center">
<h1 align="center">■ VisionGuard AI</h1>
<p align="center">Enterprise Factory Safety Monitoring & Compliance Platform</p>
<p align="center">





</p>
</p>
```

VisionGuard AI is a production-ready, AI-powered factory safety monitoring platform. It monitors factory camera feeds in real time, detects PPE violations (helmets, vests, gloves, shoes), tracks zone occupancy, identifies unauthorized access, and instantly alerts supervisors — all from a centralized web dashboard.

Built as a **Modular Monolith** following **Clean Architecture** and **Domain Driven Design** — designed so every module can be independently extracted into a microservice in the future without touching business logic.

Table of Contents

- [Features](#features)
 - [Architecture](#architecture)
 - [Tech Stack](#tech-stack)
 - [Project Structure](#project-structure)
 - [Prerequisites](#prerequisites)
 - [Quick Start](#quick-start)
 - [Environment Variables](#environment-variables)
 - [Services & Ports](#services--ports)
 - [API Overview](#api-overview)
 - [User Roles](#user-roles)
 - [AI Detection](#ai-detection)
 - [Alert System](#alert-system)
 - [Monitoring](#monitoring)
 - [Scaling](#scaling)
 - [Roadmap](#roadmap)
 - [License](#license)
-

Features

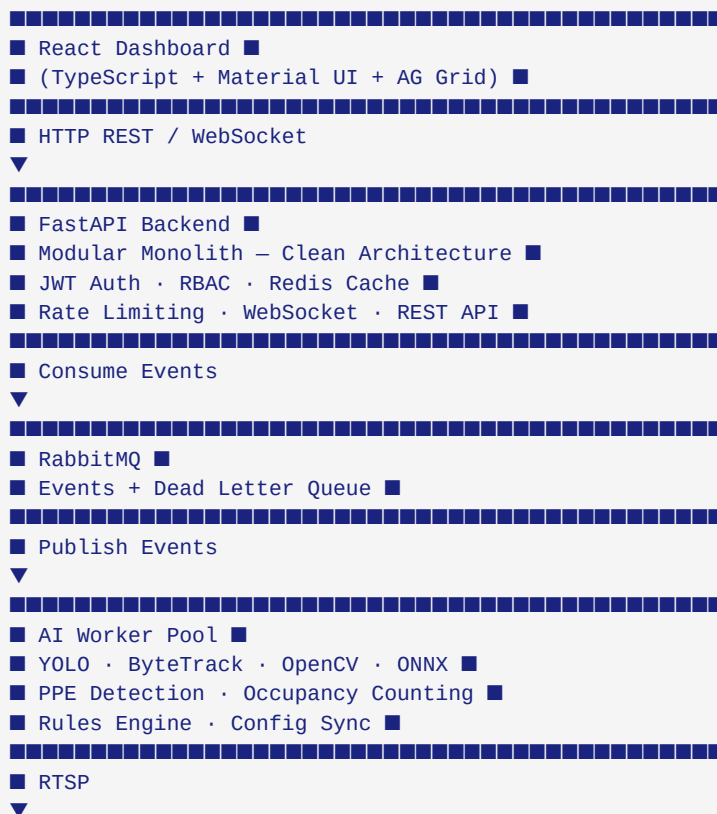
Safety Monitoring

- **PPE Detection** — helmet, safety vest, gloves, shoes per zone
- **Occupancy Monitoring** — real-time person count with zone-level capacity rules
- **Unauthorized Entry Detection** — restricted zone access alerts
- **Camera Health Monitoring** — auto-detects offline cameras within 60 seconds

Platform

- **Real-Time Dashboard** — WebSocket-driven, zero polling
- **Instant Alerts** — < 5 second latency from detection to notification
- **Violation Snapshots** — stored in MinIO, served via pre-signed URLs
- **Audit Trail** — every action logged immutably for compliance
- **Role Based Access Control** — Admin, Safety Officer, Supervisor, Viewer
- **Zone Config Hot-Swap** — change detection thresholds at runtime, no restart needed
- **AI Model Hot-Swap** — update models without stopping camera streams
- **Analytics & Reports** — daily/weekly/monthly trends, compliance %, export to PDF/Excel
- **Scales to 200+ cameras** — horizontal scaling with no rewrite

Architecture



[illegible]

> AI Workers **never access the database directly** — they only publish events to RabbitMQ. The backend consumes events, writes to PostgreSQL, invalidates Redis cache, and pushes real-time updates via WebSocket.

Full architecture → [IMPLEMENTATION.md](./IMPLEMENTATION.md)

Tech Stack

Layer	Technology
Frontend	React 18, TypeScript, Vite, Material UI, AG Grid, Recharts, TanStack Query, Zustand, React Hook Form, Zod
Backend	FastAPI, SQLAlchemy 2.0, Alembic, Pydantic v2, PostgreSQL 16
AI	YOLO, OpenCV, ByteTrack, ONNX Runtime, PyTorch (CUDA + CPU fallback)
Messaging	RabbitMQ with Dead Letter Queue
Cache	Redis
Storage	MinIO (S3-compatible)
Monitoring	Prometheus, Grafana, Loki
Infrastructure	Docker, Docker Compose, Nginx
CI/CD	GitHub Actions

All open source. Zero licensing cost for self-hosted deployment.

Project Structure

```
visionguard-ai/
■
■■■ backend/ FastAPI Modular Monolith
■ ■■■ src/
■ ■■■ main.py Application entrypoint
■ ■■■ core/ App factory, lifespan, exception handlers
■ ■■■ shared/ Cross-cutting concerns
■ ■■■ database/ SQLAlchemy engine, session, base model
```

- ■ ■■ cache/ Redis client + cache helpers
- ■ ■■ realtime/ WebSocket connection manager
- ■ ■■ messaging/ RabbitMQ consumer
- ■ ■■ middleware/ Auth, logging, correlation ID, rate limit
- ■■ modules/ Business modules (Clean Architecture)
- ■■ identity/ Auth, JWT, refresh tokens, users, roles
- ■■ organization/ Multi-tenant organization management
- ■■ factory/ Factory CRUD
- ■■ zone/ Zone management + occupancy rules
- ■■ camera/ Camera registry + health monitoring
- ■■ worker/ AI worker registry + heartbeat
- ■■ occupancy/ Real-time occupancy tracking
- ■■ ppe/ PPE violation records
- ■■ alerts/ Alert lifecycle management
- ■■ notifications/ Email, in-app, web push, Slack, Teams
- ■■ analytics/ Trends, KPIs, heatmaps
- ■■ dashboard/ Dashboard aggregation APIs
- ■■ reports/ PDF + Excel report generation
- ■■ config/ Zone-level config pushed to AI workers
- ■■ audit/ Immutable compliance audit log
- ■■ health/ Readiness + liveness endpoints
-
- ■■ ai-worker/ Python AI Detection Workers
- ■■ src/
- ■■ main.py Worker entrypoint
- ■■ pipeline/ CameraWorker – full detection loop
- ■ ■■ frame_reader.py RTSP capture + reconnect
- ■ ■■ detector.py YOLO inference
- ■ ■■ tracker.py ByteTrack integration
- ■ ■■ ppe_validator.py PPE rule evaluation
- ■ ■■ occupancy_counter.py Person count per zone
- ■ ■■ rules_engine.py Configurable violation rules
- ■■ events/ RabbitMQ publisher
- ■■ config/ Settings + zone config sync
- ■■ health/ Worker health + metrics
-
- ■■ frontend/ React Application
- ■■ src/
- ■■ App.tsx
- ■■ store/ Zustand global state
- ■■ api/ TanStack Query hooks
- ■■ pages/
- ■ ■■ Dashboard/ Executive + Safety + Camera dashboards
- ■ ■■ Alerts/ Alert list, detail, acknowledge, resolve
- ■ ■■ Cameras/ Camera grid, health, live status
- ■ ■■ Analytics/ Charts, heatmaps, KPIs
- ■ ■■ Reports/ Generate + download reports
- ■ ■■ Config/ Zone config management
- ■ ■■ Admin/ Users, roles, factories, zones
- ■■ components/ Shared UI components
-
- ■■ infra/
- ■■ nginx/ Nginx reverse proxy config
- ■■ prometheus/ Prometheus scrape config
- ■■ grafana/ Grafana dashboards (JSON)
- ■■ loki/ Loki log aggregation config
-

```

■■■■ docs/
■ ■■■■ ARCHITECTURE.md System architecture diagrams
■ ■■■■ API.md Full API reference
■ ■■■■ DATABASE.md Schema + ER diagram
■ ■■■■ DEPLOYMENT.md Production deployment guide
■
■■■■ docker-compose.yml Development environment
■■■■ docker-compose.prod.yml Production environment
■■■■ .env.example Environment variable template
■■■■ README.md
■■■■ IMPLEMENTATION.md Full technical specification

```

Every backend module follows this internal structure:

```

module/
■■■■ api/ REST endpoints + WebSocket handlers
■■■■ application/ Use cases, command handlers, query handlers
■■■■ domain/ Entities, value objects, domain events, business rules
■■■■ infrastructure/ SQLAlchemy models, repositories, external clients
■■■■ tests/ Unit + integration tests

```

Prerequisites

Tool	Version	Purpose
[Docker Desktop](https://www.docker.com/products/docker-desktop/)	Latest	Run all infrastructure
[NVIDIA Container Toolkit](https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/)	Latest	GPU inference
[Node.js](https://nodejs.org/)	20+	Frontend dev
[Python](https://www.python.org/)	3.11+	Backend + AI worker

> Windows: Docker Desktop with WSL2 required for GPU passthrough.

Quick Start

1. Clone

```

git clone https://github.com/Mithunkiet/visionguard-ai.git
cd visionguard-ai

```

2. Configure

```
cp .env.example .env
# Edit .env – fill in passwords and secrets
```

Generate secrets:

```
openssl rand -hex 32 # use for JWT_SECRET_KEY
openssl rand -hex 32 # use for REFRESH_TOKEN_SECRET
```

3. Start infrastructure

```
docker compose up postgres rabbitmq redis minio -d
```

4. Start backend

```
cd backend
pip install -r requirements.txt
alembic upgrade head # run DB migrations
uvicorn src.main:app --reload --port 8000
```

API docs: <http://localhost:8000/docs>

5. Start AI worker (CPU mode for dev)

```
cd ai-worker
pip install -r requirements.txt
DEVICE=cpu python -m src.main
```

> For real cameras: add RTSP URLs to `ai-worker/src/config/settings.py` → CAMERAS list.

6. Start frontend

```
cd frontend
npm install
npm run dev
```

Frontend: <http://localhost:5173>

Start Everything with Docker

```
docker compose up --build
```

Production:

```
docker compose -f docker-compose.prod.yml up --build -d
```

Environment Variables

Variable	Description	Example
<code>`POSTGRES_HOST`</code>	PostgreSQL host	<code>`localhost`</code>
<code>`POSTGRES_DB`</code>	Database name	<code>`visionguard`</code>
<code>`POSTGRES_USER`</code>	DB username	<code>`visionguard`</code>
<code>`POSTGRES_PASSWORD`</code>	DB password	<code>*(strong password)*</code>
<code>`RABBITMQ_HOST`</code>	RabbitMQ host	<code>`localhost`</code>
<code>`RABBITMQ_USER`</code>	RabbitMQ username	<code>`visionguard`</code>
<code>`RABBITMQ_PASSWORD`</code>	RabbitMQ password	<code>*(strong password)*</code>
<code>`REDIS_URL`</code>	Redis connection URL	<code>`redis://localhost:6379`</code>
<code>`MINIO_ENDPOINT`</code>	MinIO endpoint	<code>`localhost:9000`</code>
<code>`MINIO_ROOT_USER`</code>	MinIO root user	<code>`visionguard`</code>
<code>`MINIO_ROOT_PASSWORD`</code>	MinIO root password	<code>*(strong password)*</code>
<code>`JWT_SECRET_KEY`</code>	JWT signing secret	<code>*(32-byte hex)*</code>
<code>`REFRESH_TOKEN_SECRET`</code>	Refresh token secret	<code>*(32-byte hex)*</code>
<code>`DEVICE`</code>	AI inference device	<code>`cuda`</code> or <code>`cpu`</code>
<code>`WORKER_ID`</code>	Unique worker instance ID	<code>`worker-1`</code>

See `.env.example` for the full list.

Services & Ports

Service	URL	Default Credentials
Frontend	<code>`http://localhost:5173`</code>	—
Backend API	<code>`http://localhost:8000`</code>	—
API Docs (Swagger)	<code>`http://localhost:8000/docs`</code>	—
RabbitMQ Console	<code>`http://localhost:15672`</code>	<code>`visionguard / visionguard123`</code>
MinIO Console	<code>`http://localhost:9001`</code>	<code>`visionguard / visionguard123`</code>
Grafana	<code>`http://localhost:3000`</code>	<code>`admin / admin`</code>
Prometheus	<code>`http://localhost:9090`</code>	—
PostgreSQL	<code>`localhost:5432`</code>	—
Redis	<code>`localhost:6379`</code>	—

> Change all default credentials before production deployment.

API Overview

```
# Auth
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout

# Factories & Zones
GET /api/v1/factories
POST /api/v1/factories
GET /api/v1/zones
POST /api/v1/zones

# Cameras
GET /api/v1/cameras
POST /api/v1/cameras
GET /api/v1/cameras/{id}/health

# Alerts
GET /api/v1/alerts
GET /api/v1/alerts/{id}
PATCH /api/v1/alerts/{id}/acknowledge
PATCH /api/v1/alerts/{id}/resolve

# Analytics
GET /api/v1/analytics/violations
GET /api/v1/analytics/occupancy
GET /api/v1/analytics/compliance
GET /api/v1/analytics/safety-score

# Reports
POST /api/v1/reports/generate
GET /api/v1/reports/{id}/download

# Zone Config
GET /api/v1/config/zone/{zone_id}
PUT /api/v1/config/zone/{zone_id}

# Audit
GET /api/v1/audit

# WebSocket
WS /ws/dashboard
WS /ws/alerts
```

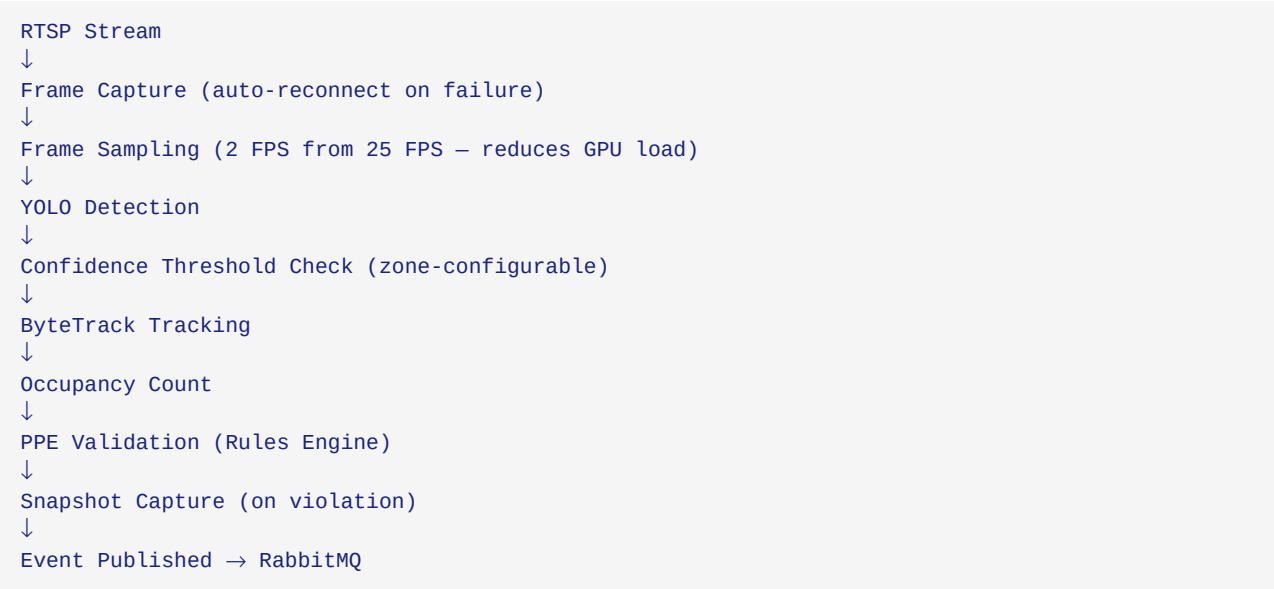
Full API reference → [[docs/API.md](#)](./docs/API.md)

User Roles

Role	Cameras	Alerts	Config	Users	Audit	Reports
Admin	Full	Full	Full	Full	Full	Full
Safety Officer	View	Investigate	—	—	View	Generate
Supervisor	View	Ack + Resolve	—	—	—	View
Viewer	View	View	—	—	—	—

AI Detection

Detection Pipeline (per camera)



Default Confidence Thresholds

Detection	Default Threshold
Person	0.70
Helmet	0.75
Safety Vest	0.75
Gloves	0.70
Safety Shoes	0.70

All thresholds are configurable per zone at runtime — no restart required.

Camera Resilience

Stream lost → retry in 5s → retry in 15s → retry in 60s
→ still offline → publish CameraOfflineDetected event
→ backend creates alert → IT team notified

3 consecutive processing failures → camera isolated (circuit breaker) → worker continues other cameras.

Alert System

Severity Levels

Severity	Examples
Critical	Fire, explosion, fall detection
High	Helmet missing, overcrowding, restricted zone entry
Medium	Vest missing, gloves missing
Low	Camera FPS drop, stream delay

Alert Lifecycle

Open → Acknowledged → In Progress → Closed

Every status change is logged to `alerts.alert_history` with timestamp, user, and comment.

Events (RabbitMQ)

Event	Routing Key
Helmet missing	<code>`events.helmet_missing_detected`</code>
Vest missing	<code>`events.vest_missing_detected`</code>
Overcrowding	<code>`events.overcrowding_detected`</code>
Unauthorized entry	<code>`events.unauthorized_entry_detected`</code>
Camera offline	<code>`events.camera_offline_detected`</code>
Camera reconnected	<code>`events.camera_reconnected`</code>
Occupancy updated	<code>`events.occupancy_updated`</code>
Worker heartbeat	<code>`events.worker_heartbeat`</code>

Failed events → Dead Letter Queue → auto-retry (3 attempts) → manual review queue.

Monitoring

```
docker compose up prometheus grafana loki -d
```

Grafana Dashboards (<http://localhost:3000>):

Dashboard	What it shows
System Overview	API health, error rate, latency
AI Worker Health	Inference latency (p50/p90/p99), frame rate, queue depth
Alert Operations	Alert volume, resolution time, open alerts by zone
Camera Health	Online/offline count, reconnection rate
DLQ Monitor	Failed event depth, retry rate

Scaling

Cameras	Backend	AI Workers	Notes
50	1 server	2 GPU workers	Initial target
100	1 server	4 GPU workers	
200	2 servers	8 GPU workers	Redis pub/sub for WebSocket
500	3–5 node cluster	20+ workers	PostgreSQL read replicas
1000+	API Gateway + cluster	50+ workers	Kubernetes

No code changes required — horizontal scaling only.

Roadmap

Phase 1 (Current)

- Helmet + vest detection
- Occupancy monitoring
- Real-time alerts
- Core dashboard

Phase 2

- Face recognition
- Visitor tracking
- Attendance integration

- Gloves + shoes detection

Phase 3

- Fire & smoke detection
- Fall detection
- SAP / HRMS integration

Phase 4

- Forklift monitoring
- Unsafe behavior detection
- Predictive safety analytics
- AI Risk Scoring
- Digital safety audits

License

MIT License — see [LICENSE](./LICENSE) for details.

All dependencies are open source. See [IMPLEMENTATION.md](./IMPLEMENTATION.md) for per-component license details.

Enterprise Hierarchy

VisionGuard AI follows a 5-level enterprise hierarchy:

```
Enterprise (e.g. {Enterprise Name})
  ■■■ Factory (AC Factory / WM Factory / Fridge Factory)
    ■■■ Department (Assembly / Welding / QC)
      ■■■ Zone (Zone A / Zone B)
        ■■■ Camera (CAM-001 / CAM-002)
```

This allows a single platform deployment to centrally monitor all factories of an enterprise, while keeping each factory's data completely isolated from others.

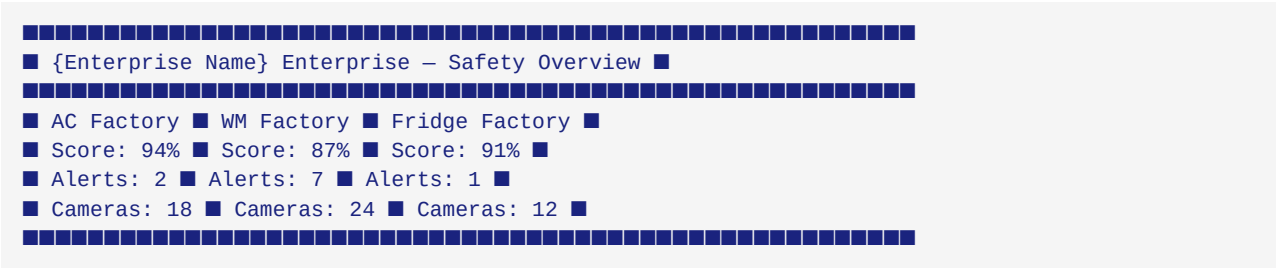
Multi-Factory Access Control

Role	Scope	What They See
HO Admin	Enterprise-wide	All factories, all data, cross-factory analytics

Role	Scope	What They See
Factory Admin	Single factory	Only their factory — zones, cameras, alerts, users
Department Head	Single department	Only their department's zones and cameras
Safety Officer	Single factory	Factory alerts, violations, reports
Supervisor	Assigned zones	Only their assigned zones
Viewer	Assigned scope	Read-only dashboard

> A Factory Admin of AC Factory will **never** see data from WM Factory — enforced at middleware level on every API request.

Cross-Factory Enterprise Dashboard (HO Admin Only)



HO Admin can drill down into any factory from this view.

Supply to Multiple Enterprises

VisionGuard AI is designed to be supplied to multiple enterprise clients:

```
Client 1 → {Enterprise Name} (4 factories, 200 cameras)
Client 2 → {Enterprise Name 2} (6 factories, 350 cameras)
Client 3 → {Enterprise Name 3} (3 factories, 150 cameras)
```

Each enterprise is a completely isolated tenant. One enterprise cannot see another enterprise's data — enforced via `enterprise_id` scoping on every database query.

Dynamic Branding

VisionGuard AI contains **zero hardcoded company names or logos** anywhere in the platform.

All branding is loaded dynamically from the database at runtime:

Element	Source
Company name (UI, emails, PDFs)	`enterprise.name` from DB
Logo (header, login, reports)	`enterprise.logo_url` → MinIO
UI theme color	`enterprise.primary_color` from DB
Browser tab title	`{enterprise.name} — VisionGuard AI`
Export filenames	`{enterprise.code}_report_2026.pdf`
Email subjects	`[{enterprise.name}] Alert Notification`

Each enterprise client sees their own branding automatically — the platform adapts to whoever is logged in.

First Time Setup — Onboarding Flow

Phase 1 — Super Admin (VisionGuard Team)

1. Super Admin logs in (MFA required)
 2. Creates enterprise account (name, logo, colors)
 3. Creates first HO Admin user
 4. System sends Welcome Email with temporary password

Phase 2 — HO Admin (Client)

5. HO Admin receives Welcome Email
 6. Logs in → forced password change
 7. Auto-redirected to Setup Wizard

Setup Wizard (4 Steps)

- Step 1 → Create Factory (name, code, location)

Step 2 → Add Department (name, department head)

Step 3 → Configure Zone (max occupancy, PPE rules)

Step 4 → Connect Camera (RTSP URL + test connection)

↓

■ Dashboard goes live

> Setup Wizard progress is saved at each step — if admin closes browser midway, they resume from where they left off.

Complete Feature List

Security

- JWT + Refresh Token Rotation
- Two Factor Authentication (2FA/TOTP)
- Role Based Access Control (RBAC)
- Session Timeout per role
- IP Whitelisting per enterprise
- Password expiry + history
- Token blacklisting on logout
- Account lockout (5 failed attempts)
- Immutable audit trail

AI Detection

- Helmet, Vest, Gloves, Shoes, Mask detection
- Real-time occupancy counting
- Unauthorized zone entry detection
- ByteTrack multi-person tracking
- Configurable confidence thresholds per zone
- AI model hot-swap (no stream restart)
- False positive marking + tracking
- Circuit breaker per camera

Alert Management

- Full lifecycle: Open → Acknowledged → In Progress → Closed
- 5-level escalation matrix (configurable)
- Alert deduplication + cooldown
- Alert snooze per camera
- Bulk alert actions (ack/assign/close)
- False positive marking
- Resolution note templates
- SLA tracking per severity

Dashboard

- Executive Dashboard (HO Admin — all factories)
- Safety Dashboard (Safety Officer)
- Zone Dashboard (Supervisor)
- Camera Dashboard
- Worker Dashboard
- Real-time Activity Feed
- Full screen mode (control room)
- Dark mode
- WebSocket-driven (zero polling)

Notifications

- Email, In-App, Web Push, Slack, Teams, Webhook, SMS
- Notification digest (15min / 1hr / daily)

- Do Not Disturb hours
- Per-user channel preferences per severity
- Read/unread tracking
- Retry on delivery failure

Reports

- Daily, Weekly, Monthly (auto-scheduled)
- Custom date range
- Comparative (this month vs last month)
- Zone-wise, Camera-wise, Shift-wise
- PDF + Excel export
- CSV raw data export
- Automated email delivery

Configuration

- Zone-level PPE detection config
- Fully configurable rules engine (duration + cooldown)
- Config history + restore
- Config copy between zones
- Config templates
- Bulk config update
- All changes hot-applied to AI workers (no restart)

Shift Management

- Morning / Evening / Night shift configuration
- Shift-wise violation analytics
- Peak violation shift identification
- Shift-wise reports

Camera Maintenance

- Maintenance mode (no alerts during maintenance)
- Preventive maintenance scheduling
- Maintenance history per camera
- Overdue maintenance warnings

User Management

- Bulk user import via CSV
- User groups + group notifications
- Temporary access with auto-expiry
- User activity log
- 2FA enforcement per role

Announcements

- Enterprise / Factory / Department scope
- Urgent banners + normal notice board

- Read receipts per user
- Auto-expiry

Client API Access

- API keys with configurable permissions
- Webhook support for external integrations
- API usage logging
- Instant key revocation

Dynamic Branding

- Enterprise name, logo, colors from database
- Zero hardcoded company names anywhere
- Per-enterprise theme color
- Dynamic PDF/email branding

Onboarding

- Super Admin creates enterprise + first admin
- Setup Wizard (4 steps: Factory → Dept → Zone → Camera)
- RTSP connection test
- Progress saved — resume if browser closed
- Auto AI worker assignment on completion